University of Cape Town
Department of Computer Science

Computer Science CSC3003S
Class Test 2, 2021

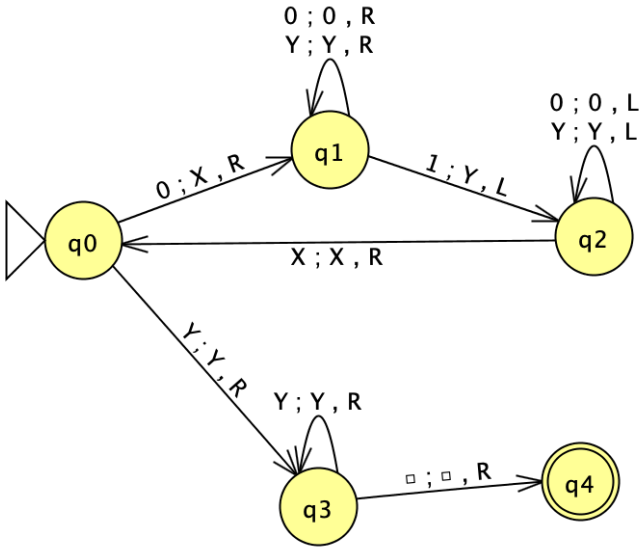
Answer all questions.
Approximate marks per question are shown in brackets.

Time: 40 minutes **Marks: 40**

Section 1: Theory of Computation

Question 1. **[3]**

Consider the following Turing Machine for some language or decision problem that accepts by final state, as depicted in the figure below.



a) Show the computation of the input string 011, using the notation of *instantaneous descriptions*. [1.5]

b) Describe the situation that the Turing Machine is in when it is at that last instantaneous description. [1.5]

a) $q_0011 \vdash Xq_111 \vdash q_2XY1 \vdash Xq_0Y1 \vdash XYq_31$ (and then no more moves are possible)
b) the TM halts in a non-final state and so rejects the input (alternative terminology that is also fine: it dies/stops, and that it will answer 'no' to the question whether the input string is in the language that the TM is designed for)

Question 2. **[7]**

Design a basic – i.e., one tape, one track, deterministic – Turing Machine that, given some sequence of 0s and 1s as input string on its tape, deletes the first symbol (i.e., a 0 or a 1) and shifts all the following 0's and 1's one position to the left, and then halts. For instance, and with B the blank symbol, if there's some piece of tape with just the input on it at the beginning, …BB1010BB…, then it should halt with …BB010BB… on its tape.

Provide the *full definition* of the TM. The transition function may be drawn (using consistent common notation), listed in a list of functions (adhering to the definition of the function), or shown in a table with the states (first column the states, first row the alphabet).

ANSWER - begin

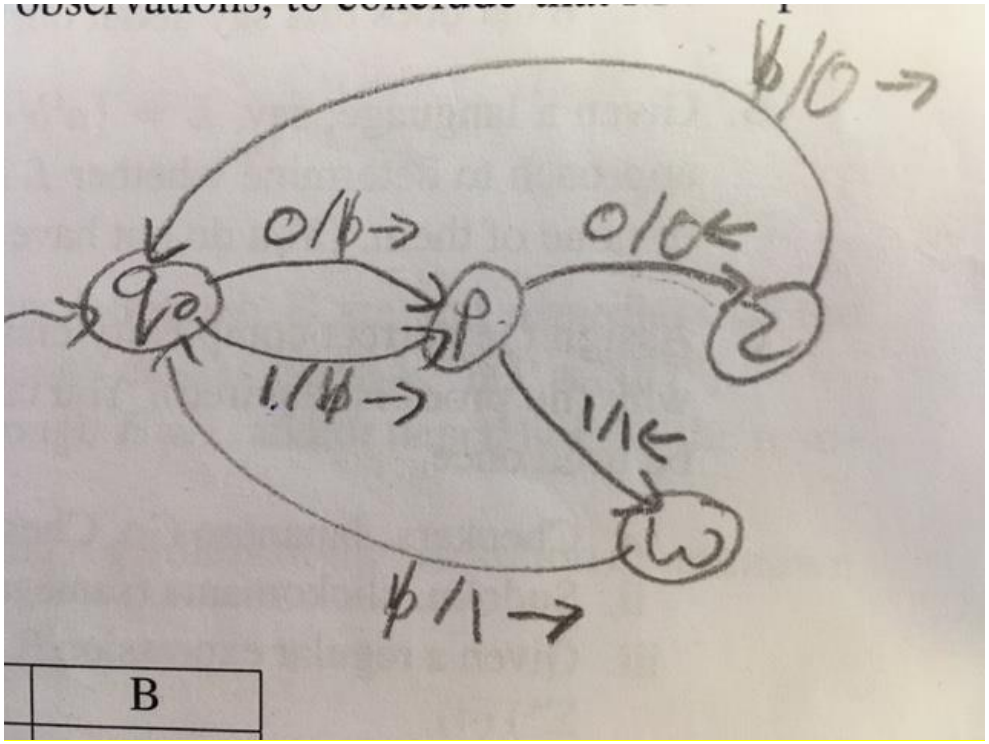
$M = (Q, \Sigma, \Gamma, \delta, q_0, B, F)$, where:

- Q is the set of states $\{q_0, p, z, w\}$
- Σ input alphabet $\{0, 1\}$
- Γ tape alphabet $\{0, 1, B\}$
- B the blank symbol
- q_0 start state
- F set of final states
- δ transition function: see Table 2.

Table 2:

State	0	1	B
q_0	(p, B, R)	(p, B, R)	
p	$(z, 0, L)$	$(w, 1, L)$	
z			$(q_0, 0, R)$
w			$(q_0, 1, R)$

Instead of a table with the transition functions, here's my drawing:



Note for marking: either the diagram or the table, or as a list of functions (delta(q0,0) = (p,B,R) etc etc)

Marks: 2 for listing all the TM components (listed before the table above). For the remaining 5 marks: give 5 for everything correct, 2.5 if it does about half of it, 0 if wrong (accepts a different language, incomplete). Deduct 0.5 mark for inconsistent notation, 0.5 for missing the arrow/forgetting to indicate the start symbol, 0.5 for forgetting arrows in the transitions.

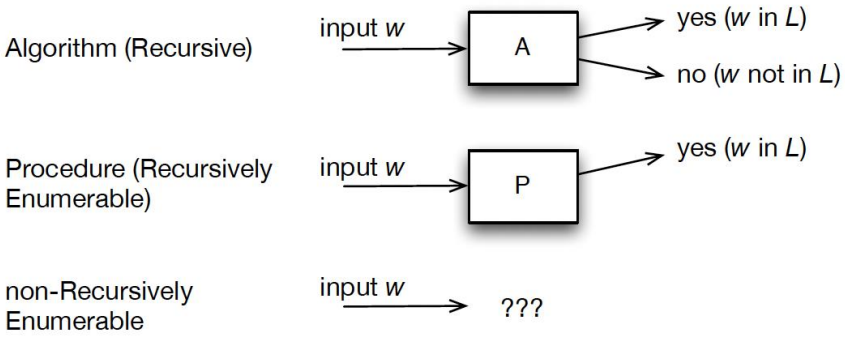
NOTE: accepting by final state is also an option though not fully according to specs. If they added a final state, then that can only be with the transition delta(q0,B) = (qf,B, R) (or L instead of R – doesn't matter). Give 4 marks in that case.

NOTE: it also can be done with a 2 tape machine, but the question says explicitly one tape, one track DTM.

Question 3. **[5]**

Sketch or describe the differences between recursive, recursively enumerable, and non-recursively enumerable languages in the context of Turing machines, and how they relate to something like the Hello World Problem specifically and to algorithms and procedures more generally.

The easiest would be to draw something like the following figure, which would get full marks when the non-RE one has the HWP there:



Alternatively, to describe that the recursive ones correspond to the algorithms, always a y or n on any given input [2]. The RE ones [that are not recursive] correspond to the procedures that can give a y but that's it [2], and non-RE gets into knots (like the HWP or Foo problem) and no TM can do anything with it [1].

Marking note: the "w in L" does not need to be represented. Input "I" and just yes/no is also fine.

Section 2: Theory of Algorithms

Question 4. **[12]**

a) Answer True or False to each assertion below and explain your answers in each case:

i) $\Theta(n \log_2 n) \subseteq \Theta(n)$ [2]

ii) if $y \in O(1)$ then $y \in O(n)$ [2]

iii) if $x \in \Omega(n!)$ then $x \in \Omega(n^2)$ [2]

i) False [1]. Big-theta is an exact bound so it is not possible for an algorithm to be in two different theta efficiency classes [1].

ii) True [1]. Big-O is an upper bound so that if an algorithm is in a better efficiency class (e.g., constant) it is also bound above by a worse efficiency class (e.g., linear) [1].

iii) True [1]. Big-Omega is a lower bound so that if an algorithm is in a worse efficiency class (e.g., factorial) it is also bound below by a better efficiency class (e.g., quadratic) [1].

c) Consider the following algorithm:

```
ALGORITHM strange(int [] A, int i, int j, int x)
  if i >= j do
      k = i + (j - i)/2
      if A[k] == x do
            return TRUE
```

```
        if A[k] > x do
            return strange(A, i, k-1, x)
        else
            return strange(A, k+1, j, x)
    else
      return FALSE
```

i) What does the algorithm above do, assuming A is a sorted array? What strategy (algorithmic pattern) does this algorithm use? [2]

ii) Assuming worst-case performance, set up a recurrence relation for the basic operations being performed and solve it using the Master Theorem (see Appendix A). Show all your work. [4]

i) It uses a binary search to determine if x is in the array A and returns TRUE if it is or FALSE otherwise [1]. This is an example of a decrease and conquer algorithm [1].

ii) Basic operation is comparison [1]. In the worst case x is not found in the list. Two comparisons are done and the size of the array is halved on each recursive step, thus:

C(n) = C(n/2) + 2 [1]

In terms of the Master Theorem a = 1, b = 2, d = 0 [1].

This fits the $a = b^d$ case so worst-case efficiency is θ(n log n) [1].

Question 5. **[7]**

Consider the following problem description:

You are provided with the following set of numbers *s ={1, 2, 5, 10, 50}* and an integer *x*. Your task is to choose numbers from the set *s* so that their sum matches *x*. The numbers chosen may repeat but you must use the fewest numbers possible as part of the summation.

a) Provide detailed pseudo-code for an algorithm that efficiently solves this problem. [5]
b) Run your algorithm for *x = 116* and show the output. [1]
c) By what name is this problem commonly known? [1]

a)

ALGORITHM makechange(int x, int [] s):

    tot = 0

    while tot != x **do** [1]

        i = 0

while i < len(s) **and** s[i] < (x-tot) **do**: [1/2 for bounds check, 1 for total check]

i= i+1 [1/2]

print s[i-1] [1]

tot = tot + s[i-1] [1]

b) output: 50+50+10+5+1 [1]

c) Change Making [1]

Question 6. **[6]**

Given the text *t* = RATATATTAT

And the pattern *p* = ATT

a) Construct the shift table for Horspool's algorithm. [2]

b) Show the steps taken when the pattern *p* is run against the text *t*. [2]

c) What algorithmic pattern/strategy is in use here? Explain your answer. [2]

a) A = 2, T = 1 [1 for each]

b)

```
RATATATTAT
ATT                  [1/2]
 ATT                 [1/2]
   ATT               [1/2]
     ATT (match)     [1/2]
```

c) Space-time Tradeoff [1]. The pattern is pre-processed to build a table (using extra space) in order to speed up the matching process (reducing time), hence trading off additional space for reduced time [1].